

Cognitive Robotics

Studienarbeit T3200

Studiengang Elektrotechnik
An der Dualen Hochschule Stuttgart
Campus Horb

Eingereicht von:

Name:	Luis Magnus Thiel	Adrian Sommer
Matrikelnr.:	8616207	9519924
Straße, Nr.:	Hugo-Bertsch-Straße 15	Hechinger Straße 40
PLZ, Ort:	72459 Albstadt	72336 Balingen

Partner:	ASSA ABLOY Sicherheitstechnik GmbH	Groz-Beckert KG
Adresse:	Bildstockstraße 20	Parkweg 2
Ort:	72458 Albstadt	72458 Albstadt

Studienjahrgang: TET2023

Bearbeitungszeitraum: 02. März 2026 bis 22. Juni 2026

Betreuer: Norman Bläse (Mercedes-Benz)

Kurzzusammenfassung

Diese Studienarbeit befasst sich mit der Optimierung und Erweiterung eines autonomen Robotersystems, bestehend aus einem Mitsubishi Movemaster EX RV-M1, einer 3D-ToF-Kamera sowie einer 2D-Industriekamera. Dabei ist das übergeordnete Ziel die Transformation des vorliegenden Aufbaus hin zu einem intelligenten System, welches durch die Implementierung von bildbasierten Sicherheitsfunktionen Potenziale für einen kollaborativen Betrieb aufzeigt.

Zunächst wird der Einfluss verschiedener Beleuchtungsbedingungen auf das Bildgewinnungssystem in einer Messreihe erforscht und mögliche Optimierungen erarbeitet. Nachfolgend gibt die Analyse der C++/C#-Softwarearchitektur Aufschluss über vorhandene Schwachstellen, wie auftretende Probleme bei der Thread-Synchronisierung und der Implementierung einer statischen Robotersteuerung. Für letzteres wird durch die Konzeptionierung einer dynamischen und geschlossenen Ansteuerung eine Alternative geschaffen.

Aufbauend auf den Erkenntnissen der Analyse und Konzeptionierung der dynamischen Ansteuerung wird ein Gesamtkonzept für die Entwicklung einer *Geschwindigkeits- und Abstandsüberwachung (SSM)* erstellt. Dieses Konzept nutzt die Fusion der 2D- und 3D-Bilddaten, um dynamische Warn- und Stoppbereiche softwareseitig zu realisieren. Die Arbeit bildet somit das Fundament für die sicherheitstechnische und steuerungstechnische Implementierung in der nachfolgenden Studienarbeit.

Abstract

This student research project deals with the optimisation and expansion of an autonomous robot system consisting of a Mitsubishi Movemaster EX RV-M1, a 3D ToF camera and a 2D industrial camera. The overarching goal is to transform the existing setup into an intelligent system that reveals the potential for collaborative operation through the implementation of image-based safety functions.

First, the influence of different lighting conditions on the image acquisition system is investigated in a series of measurements and possible optimisations are developed. Subsequently, the analysis of the C++/C# software architecture highlights existing weaknesses, such as problems occurring in thread synchronisation and the implementation of a static robot control system. For the latter, an alternative is created by designing a dynamic and closed-loop control system.

Based on the findings of the analysis and design of the dynamic control system, an overall concept for the development of a *Speed and Separation Monitoring (SSM)* system is created. This concept uses the fusion of 2D and 3D image data to implement dynamic warning and stop areas software-side. The work thus forms the foundation for the safety-related and control-related implementation in the subsequent student research project.

Selbständigkeitserklärung

Hiermit erklären wir, Luis Magnus Thiel und Adrian Sommer, dass wir diese schriftliche Studienarbeit selbständig verfasst haben. Es wurden keine anderen als die angegebenen Hilfsmittel und Quellen benutzt und alle wörtlich oder sinngemäß aus anderen Werken übernommenen Aussagen sind als solche gekennzeichnet.

Horb, den 9. Juni 2026

Ort, Datum



Unterschrift (Luis Magnus Thiel)

Horb, den 9. Juni 2026

Ort, Datum



Unterschrift (Adrian Sommer)

Inhaltsverzeichnis

1	Einleitung	1
2	Grundlagen	2
2.1	Stand der Technik: Safety-Implementation & Retrofitting .	2
2.2	Bildverarbeitung	2
2.3	Grundlagen der Informatik	2
2.3.1	Asynchrone Programmierung in CSharp - TPL . . .	2
2.3.2	Producer-Consumer-Pattern	2
2.3.3	Thread-Sicherheit & Blocking-Collection	2
2.3.4	Cancellation-Tokens	2
2.3.5	TCP-Socket & Protokoll-Parsing	2
3	Ausgangssituation/IST-Analyse	3
3.1	Architekturlimitationen des bisherigen Systems	3
3.1.1	Sequenzielle Befehlsstruktur	3
3.1.1.1	Theoretische Limitation	4
3.1.1.2	Messexperiment	5
3.1.1.3	Praktische Limitation - Laufzeitmessung mit Simulationsexperiment	5
4	Architekturoptimierung und -erweiterung	6
4.1	Befehlswarteschlange für dynamischere Robotersteuerung	6
5	Implementation der Erweiterungen	7
6	Fazit & Ausblick	8
7	Anhang	9
	Literaturverzeichnis	vii
	Abkürzungsverzeichnis	vii
	Abbildungsverzeichnis	x
	Erklärungen zur Arbeit und Hilfsmitteln	xi

1 Einleitung

2 Grundlagen

2.1 Stand der Technik: Safety-Implementation & Retrofitting

2.2 Bildverarbeitung

Filter, Algorithmen, Linien-/Kantendetektion mit OpenCV

2.3 Grundlagen der Informatik

2.3.1 Asynchrone Programmierung in CSharp - TPL

2.3.2 Producer-Consumer-Pattern

2.3.3 Thread-Sicherheit & Blocking-Collection

Blocking-Collection, ...

2.3.4 Cancellation-Tokens

2.3.5 TCP-Socket & Protokoll-Parsing

3 Ausgangssituation/IST-Analyse

3.1 Architekturlimitationen des bisherigen Systems

3.1.1 Sequenzielle Befehlsstruktur

Die vorliegende Befehlsübermittlung an den Movemaster EX RV-M1 wird in der Software *Objekterkennung_RoboETH* sequenziell realisiert. 3.1 zeigt qualitativ auf, wie eine solche Abfolge aussieht. Es ist

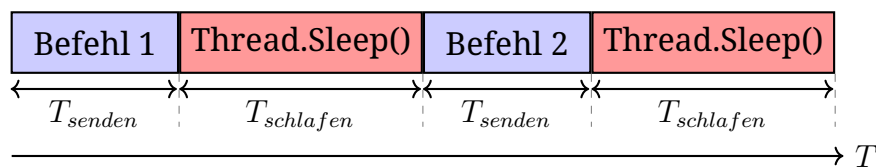


Abbildung 3.1: Sequenzielle Abfolge an Befehlen (Eigene Darstellung)

erkennbar, dass nach jedem gesendeten Befehl ein *Thread.Sleep()-Befehl* den gesamten Hauptthread für eine nicht vernachlässigbare Zeitspanne T_{schlafen} blockiert. Die theoretische Zeit T die benötigt wird, bis ein Befehl vollständig an den Roboter gesendet, von ihm verarbeitet und ausgeführt wird, lässt sich vereinfacht mit $T_{\text{befehlsausfuehrung}} = T_{\text{senden}} + T_{\text{schlafen}}$ beschreiben.

Während dieser Zeitspanne friert die gesamte Benutzeroberfläche sowie die Handlungsfähigkeit des Hauptthreads ein. Um die Auswirkungen der sequenziellen Befehlsbereitstellung auf die Gesamtlaufzeit und die Systemstabilität zu quantifizieren, ist eine detaillierte Analyse der Ausführungszeiten sowie der Thread-Verteilung erforderlich. Hierfür wird ein reproduzierbares Messexperiment aufgesetzt - hierzu nachfolgend mehr.

Das Ziel dieser Analyse ist die Identifikation der kritischen Pfade. Durch eine explizite Messung der Zeitstempel für den Eintritt in die Befehlsverarbeitung, die Dauer der Hardware-Kommunikation und die Verweildauer in den Wartezuständen wird der „Flaschenhals“

der synchronen Architektur isoliert. Erst durch diese messtechnische Herleitung wird transparent, an welcher Stelle die CPU-Last ungenutzt bleibt und wie sich die Thread-Auslastung sowie die Laufzeit bei einer Umstellung auf eine asynchrone Pipeline-Architektur verbessern lässt.

3.1.1.1 Theoretische Limitation

Wird erneut 3.1 betrachtet, so fällt auf, dass die Zeit, welche benötigt wird, um die Koordinaten der einzelnen Objekte im Arbeitsbereich des Roboters zu berechnen, nicht eingerechnet wird. Grund hierfür ist die Annahme, dass die Berechnungszeit in beiden Fällen (sequenzielle Befehlsstruktur oder asynchrone Befehlspipeline) voraussichtlich gleich ist.

Die theoretische Gesamtlaufzeit T_s einer Befehlssequenz lässt sich durch Summation der Befehlsausführungszeiten $T_{befehlsausfuehrung}$ bestimmen:

$$T_s = \sum_{i=1}^n \sum_{j=1}^k T_{befehlsausfuehrung,i,j} \quad (3.1)$$

Einsetzen der zuvor beschriebenen Zeiten ergibt:

$$T_s = \sum_{i=1}^n \sum_{j=1}^k (T_{senden,i,j} + T_{schlafen,i,j}) \quad (3.2)$$

Hierbei repräsentiert der Index i das jeweilige Objekt innerhalb der Bearbeitungsreihenfolge (mit n als Gesamtzahl der Objekte), während der Index j die spezifische Befehlsposition innerhalb eines Manipulationszyklus angibt (mit k als Anzahl der Befehle pro Objekt).

In den Gleichungen 3.1 und 3.2 lässt sich die direkte Korrelation zwischen dem Parameter n und der durch die Bildverarbeitungsklasse erkannten Objekte erkennen. Des weiteren wird verdeutlicht, dass die Gesamtausführungszeit einer Befehlssequenz (bestehend aus $1, \dots, k$ Befehlen pro Objekt) linear von der Objektanzahl n und der aufsummierten Wartezeiten durch *Thread.Sleep()*-Befehlen abhängt.

Bei dieser theoretischen Betrachtung wird bewusst eine Worst-Case-Abschätzung mittels der fest definierten Wartezeiten getroffen, da der vorliegende Roboter und die zugehörige Software keine Bestätigung für eine abgeschlossene Bewegung liefern. Das wird in der Software durch statische Wartezeiten (erfahrungsbasiert oder alternativ grob

berechnet) kompensiert, was in der Praxis allerdings zu Einbußen der Systemeffizienz führen kann.

3.1.1.2 Messexperiment

Um die theoretisch hergeleiteten Ineffizienzen und die zeitlichen Auswirkungen der sequenziellen Befehlerteilung quantitativ nachzuweisen, wurde ein deterministisches Messexperiment aufgesetzt. Selbiges muss für Vergleichbarkeit und Bewertbarkeit auch bei nachfolgenden Erweiterungen des Systems zu Grunde gelegt werden.

3.1.1.3 Praktische Limitation - Laufzeitmessung mit Simulations-experiment

Objekt-ID	Theoretische Zeit T_S [s]	Gemessene Zeit T_{mess} [s]
Objekt 1	17,5	
Objekt 2	17,5	
Objekt 3	17,5	
Objekt 4	17,5	
Objekt 5	17,5	
Mittelwert	17,5	
Gesamtzeit		

Tabelle 3.1: Vergleich der theoretischen Laufzeit T_S mit den gemessenen Werten T_{mess}

4 Architekturoptimierung und -erweiterung

4.1 Befehlswarteschlange für dynamischere Robotersteuerung

5 Implementation der Erweiterungen

Warteschlange, Safety, Punktwolkenspeicherung, ...

6 Fazit & Ausblick

7 Anhang

Literaturverzeichnis

Abkürzungsverzeichnis

CLR	Common Language Runtime
CMOS	Complementary Metal-Oxide-Semiconducto
Cobot	Collaborative Robot
CTS	Clear to Send
DFT	Diskrete Fouriertransformation
DH	Denavit Hartenberg
DIN	Deutsches Institut für Normung
DLL	Dynamic Link Library
DSR	Data Set Ready
DTR	Data Terminal Ready
DU	Drive Unit
EN	Europäische Norm
ER	Error Read
FG	Frame Ground
FOV	Field of View
fps	frames per second (Bilder pro Sekunde)
IPK	Institut für Produktionsanlagen und Konstruktionstechnik
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LED	Leuchtdiode
LIDAR	Light Detection and Ranging
MC	Move Continuous
ML	Maschinelles Lernen
MO	Move
ms	Millisekunde
Mutex	Mutual Exclusions
MVC	Model View Controller
NC	Not Connected
P/Invoke	Platform Invocation Services
PCL	Point Cloud Library
PL	Performance Level
RANSAC	Random Sample Consensus Algorithmus
RXD	Receive Data
ROR	Radius Outlier Removal
RS-232	Recommended Standard 232
SCARA	Selective Compliance Assembly Robot Arm
SOR	Statistical Outlier Removal

SP	Speed
SSM	Speed and Separation Monitoring
TCP	Tool Center Point
TCP/IP	Transmission Control Protocol/Internet Protocol
ToF	Time of Flight
TXD	Transmit Data
WH	Where
2D	Zweidimensional
3D	Dreidimensional

Abbildungsverzeichnis

3.1	Sequenzielle Abfolge an Befehlen (Eigene Darstellung)	. . .	3
-----	---	-------	---

Erklärungen zur Arbeit und Hilfsmitteln

Verwendete Hilfsmittel

Für die Erstellung dieser Studienarbeit wurden folgende Tools eingesetzt:

- **Hardware:** SICK Visionary-T Mini-CX, Mitsubishi Movemaster EX-RV M1, uEye Industriekamera
- **Software:** OpenCV Bibliothek, Visual Studio (C++ und C#), RoboETH, drawIO (Erstellung von Grafiken), texStudio, uEye-Cockpit und IDS Kameramanager, GitHub (Kollaboratives Arbeiten und Versionsverwaltung der LaTeX-Vorlage).

KI-Tools und relevanter Einsatzbereich

Dies erfolgt im Einklang mit der entsprechenden Verordnung der DHBW (Weblink).

Tool	Art der Nutzung	Abschnitt
Google Gemini AI Student Pro License	Formatierung von LaTeX-Code, Erstellung von Blankovorlagen für Tabellen und Listen	gesamte Arbeit
Google Gemini AI Student Pro License: Deep Search-Feature	Tiefgreifende Analyse des Datenblatts des Movemaster EX RV-M1: Finden von Abschlusszeichen im RS232-String, Pin-Belegung für Hardwaregestütztes Acknowledgement über RS232, Physikalische Signalanalyse für die Implementierung einer Flow-Control; Ergebnis der "Deep Search" kann jederzeit als PDF-Dokument bereitgestellt werden	4.3.2
DeepL AI Übersetzer	Formulierung des englischsprachigen Abstracts	Abstract

Aufteilung der Kapitel

Die Erstellung der vorliegenden Studienarbeit erfolgte in gemeinsamer Kooperation. In der folgenden Tabelle ist dokumentiert, welcher Autor die primäre Urheberschaft für die jeweiligen Kapitel bzw. Abschnitte trägt:

Kapitel / Abschnitt	Kapitelnummer	Autor
Einleitung	1	Gemeinsame Bearbeitung
Robotik	??	Luis Magnus Thiel
Industrielle Bildverarbeitung	??	Adrian Sommer
Grundlagen der Informatik	??	Luis Magnus Thiel
Hardware	??	Luis Magnus Thiel
Software	??	Adrian Sommer
Messreihe	??	Luis Magnus Thiel
Optimierung der Softwarestruktur	??	Luis Magnus Thiel
Optimierung der Bildgewinnung	??	Luis Magnus Thiel
Optimierung und Verbesserungsmöglichkeiten an der Software	??	Adrian Sommer
Normative Anforderungen	??	Gemeinsame Bearbeitung
Umsetzbare Sicherheitsanforderungen	??	Gemeinsame Bearbeitung
Ansatz Geschwindigkeits- und Abstandsüberwachung	??	Adrian Sommer
Fazit und Ausblick	??	Gemeinsame Bearbeitung

Hiermit bestätigen wir, Luis Magnus Thiel und Adrian Sommer,
dass die oben gemachten Angaben der Wahrheit entsprechen.

Horb, den 9. Juni 2026



Ort, Datum

Unterschrift (Luis Magnus Thiel)

Horb, den 9. Juni 2026



Ort, Datum

Unterschrift (Adrian Sommer)

Anhang

Inhalt des digitalen Anhangs

- Anleitung zur Programmaufsetzung (Luis Thiel)
- gesamter LaTeX-Ordner